

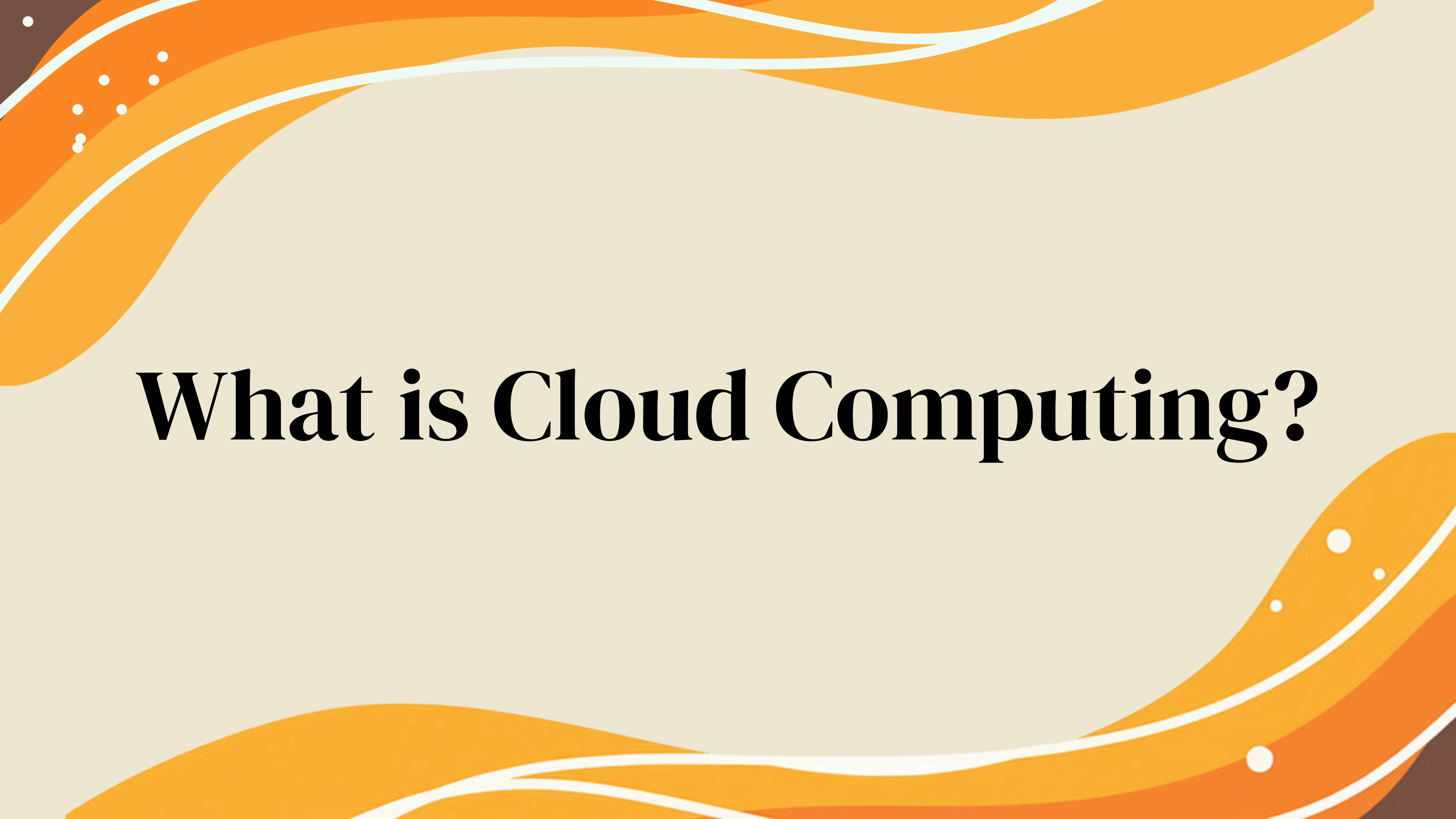
Firebase

Nada Yousry

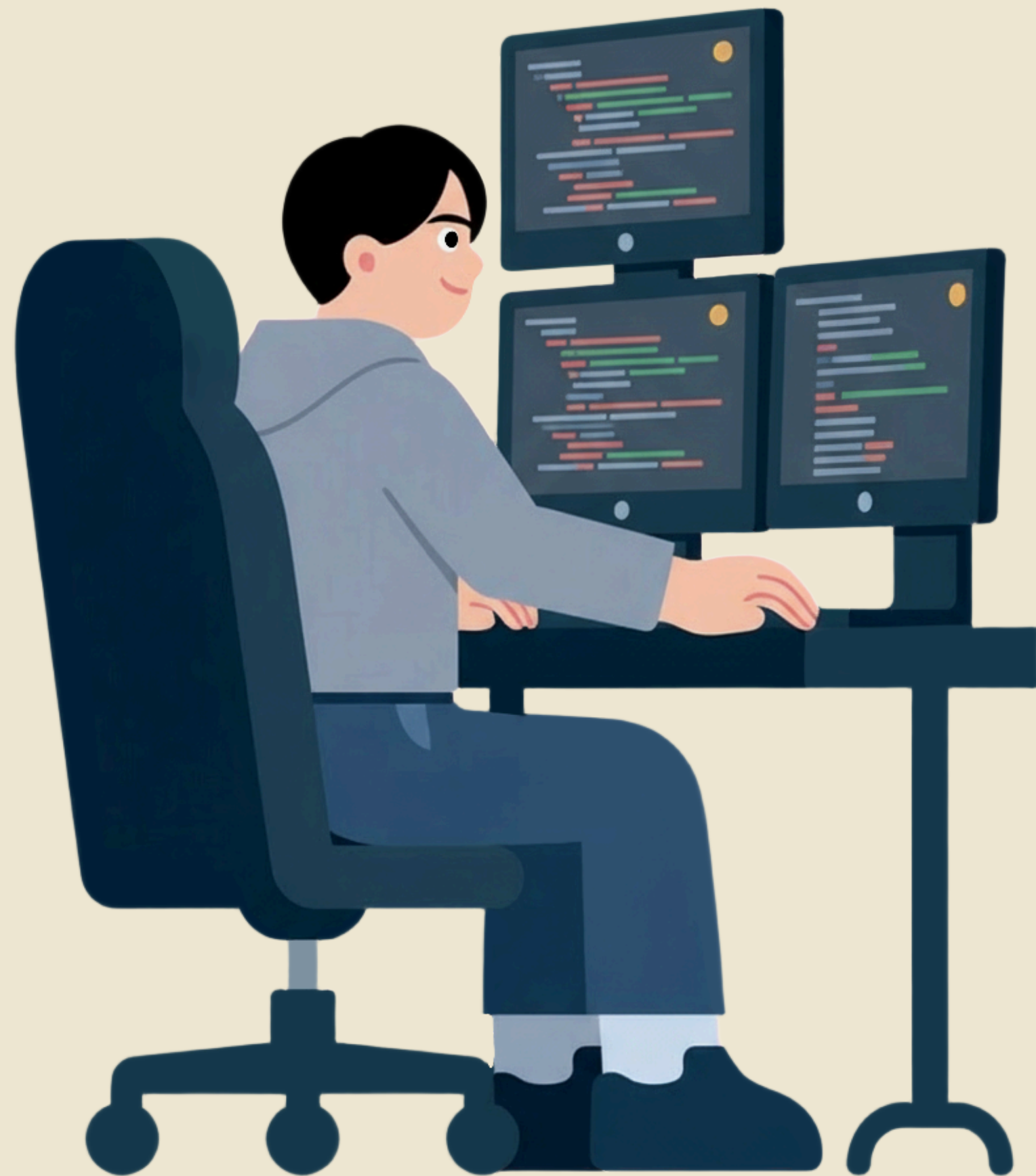
Today's Roadmap

- Introduction to Backend & Cloud Computing
- Cloud Service Models (IaaS / PaaS / SaaS / BaaS)
- Why Firebase?
- Firebase Services Overview
- Setting Up Firebase with Flutter
- Firebase Authentication
- Cloud Firestore
- CRUD Operations
- Building the Study Planner App
- Firebase Pricing (Spark vs Blaze)
- Q&A





What is Cloud Computing?



قررت ابني
ويبسايت للكلية

Offline / Development phase

Online / Deployment phase



DEPLOY



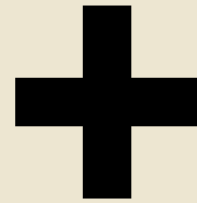
We deploy the website to make it publicly accessible,
so anyone on the internet can access it.

How to deploy website

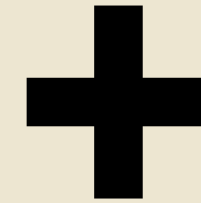
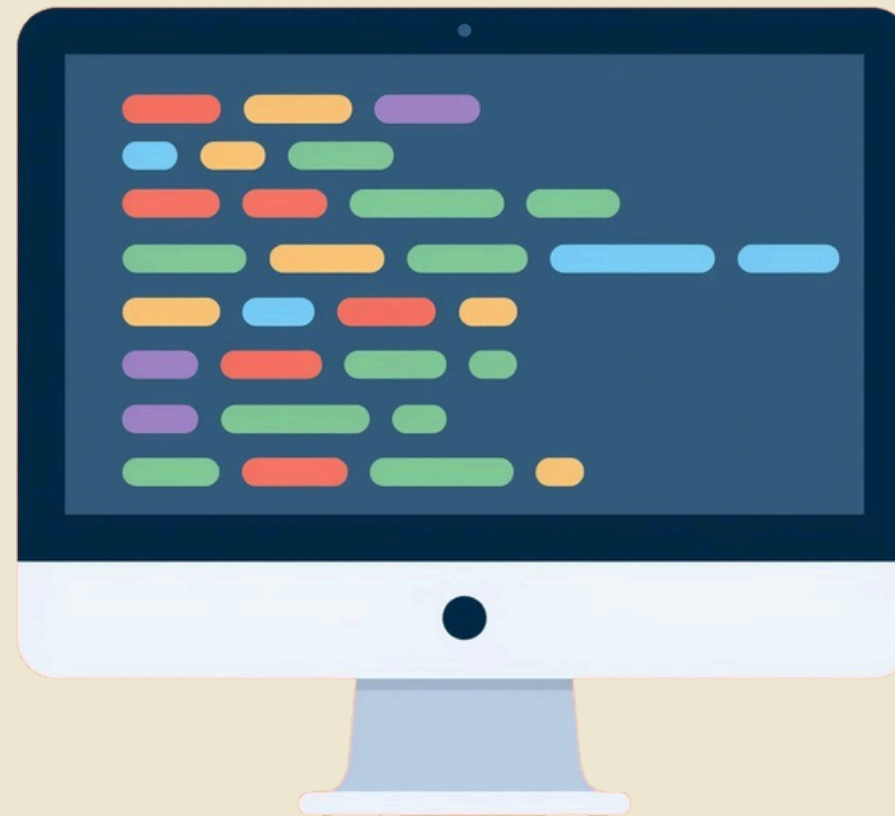
Web server



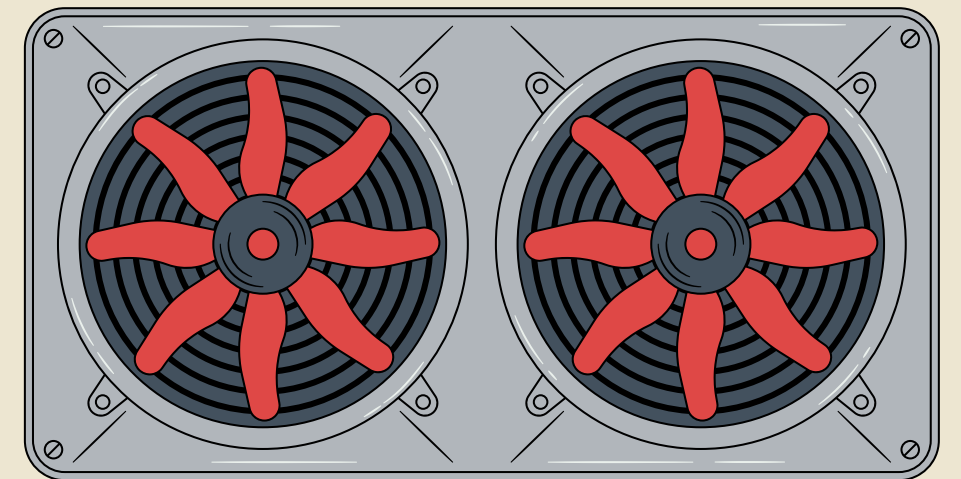
82.129.191.155



Needed Apps and
configurations



Cooling

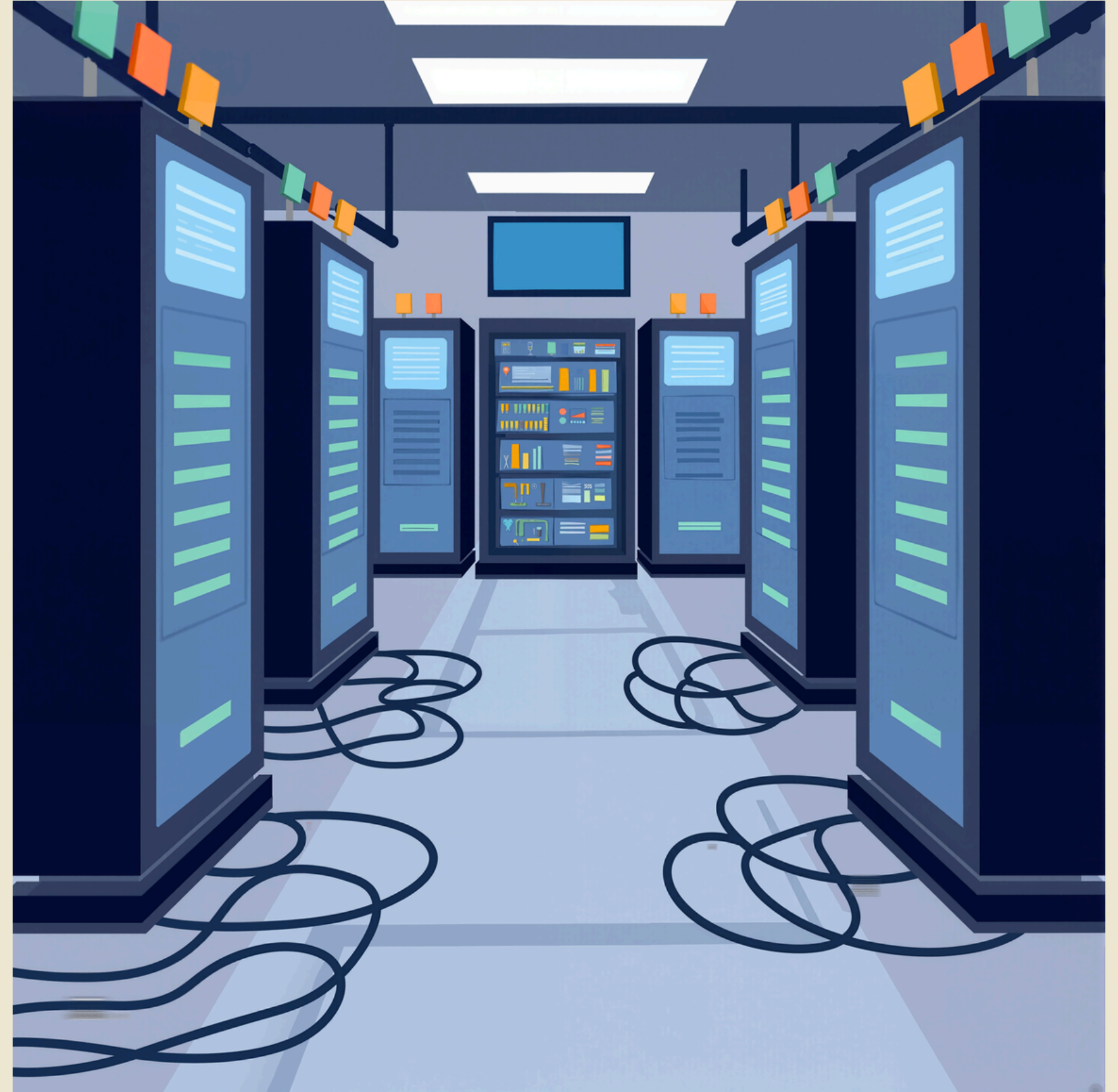


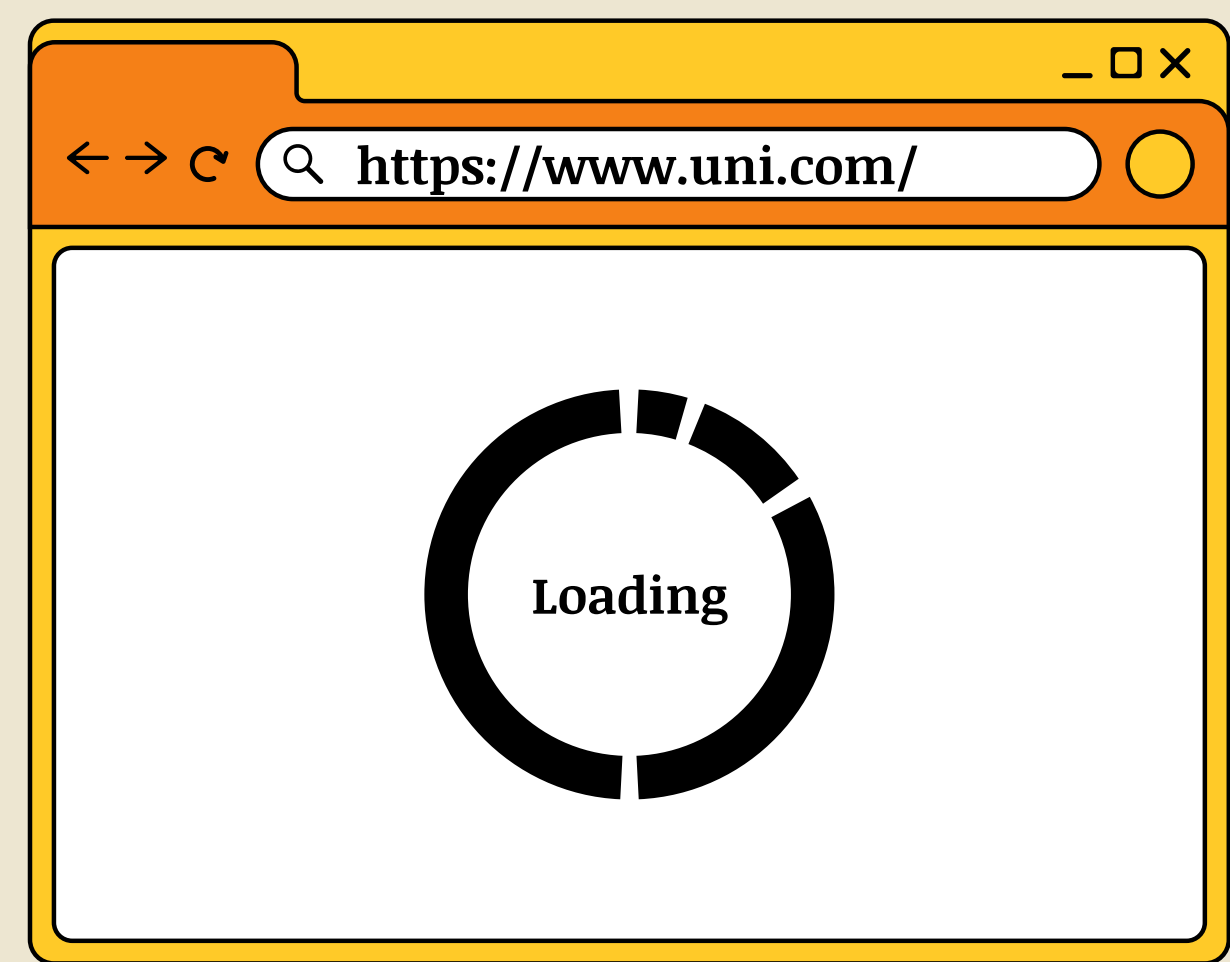
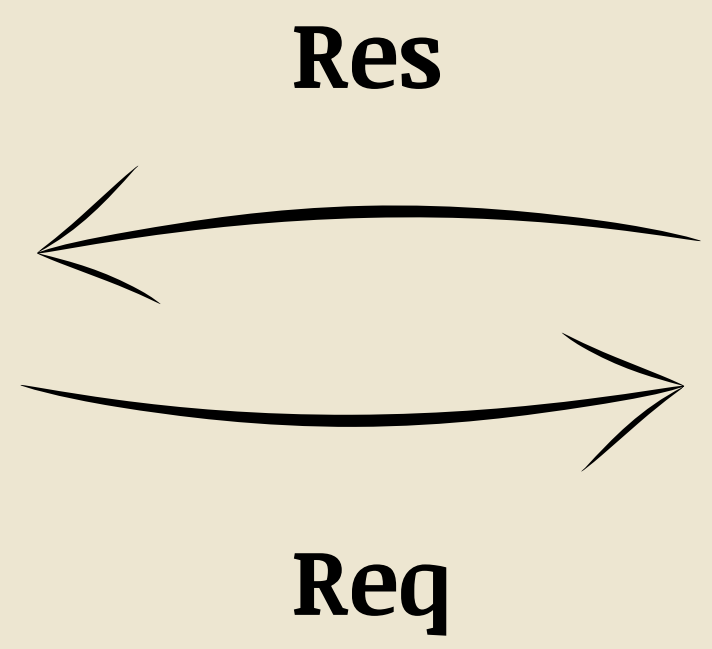
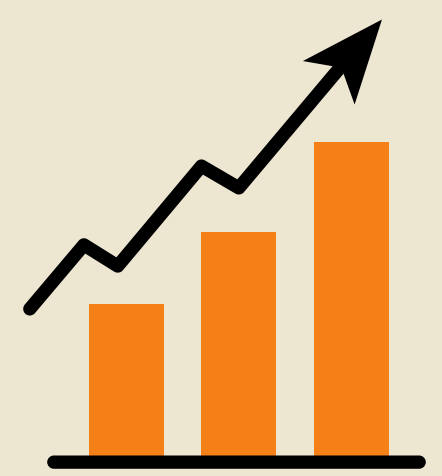
Server Room



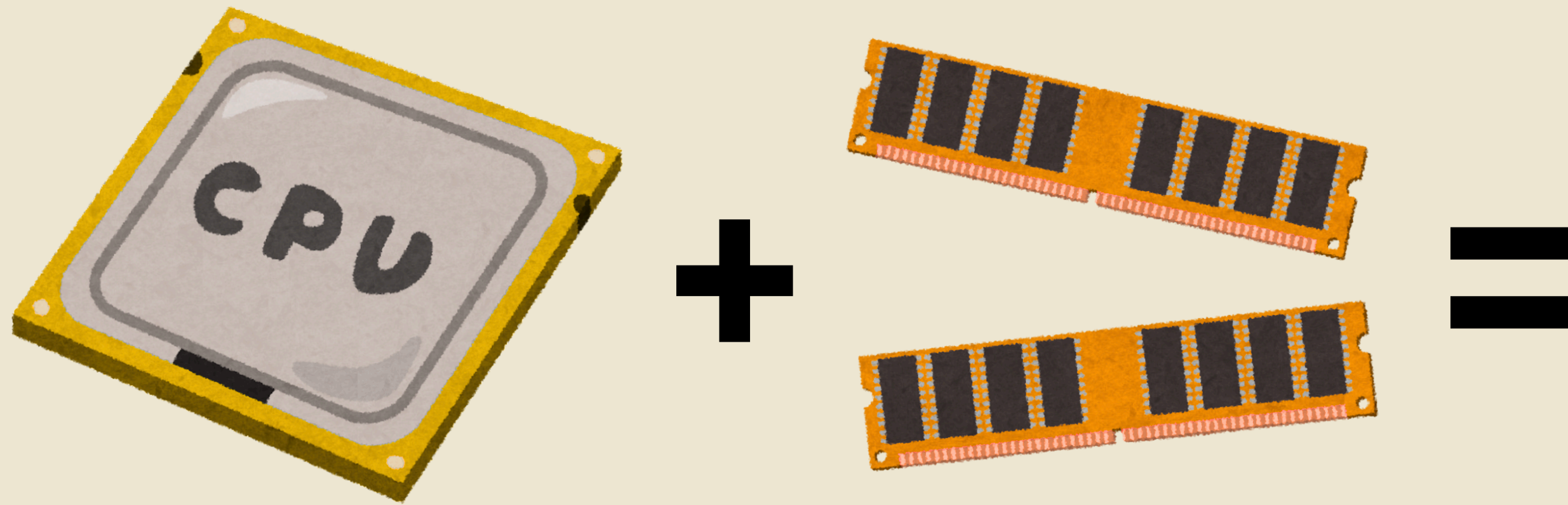
Admin

The admin manages servers and handles the necessary configurations.



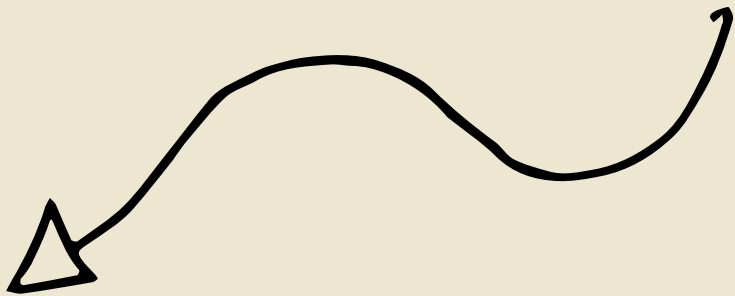
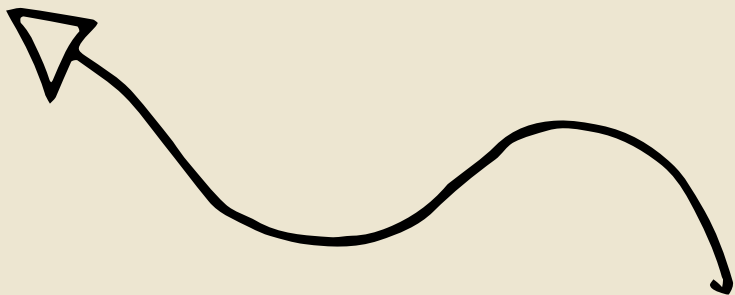


How to handel traffic



So it can handle a
large number of
concurrent users

Security



Cloud Computing

- Instead of running your own computer/server, you rent computing power, storage, and services from a company like Google, Amazon, or Microsoft
- These companies have huge data centers around the world, always on, always available

Computing Power

processing (running
code)

Storage

saving data/files,

Services

ready-made tools
(databases,
authentication, AI, etc.)

Cloud Computing Advantages



You access everything
over the internet —
from any device, any
location



Pay-as-you-go
you only pay for what
you actually use, not for
owning the hardware



Scalable on demand
It scales up automatically
— no need to buy a bigger
server yourself

Google Cloud Services

Google Cloud Services

Firestore is actually part of a bigger family: Google Cloud Platform (GCP)

GCP includes many services beyond Firestore — **compute power, big data analytics, AI/ML tools**, and more

Firestore is the layer built specifically for app developers

Today, we're focusing only on Firestore, but it's worth knowing it's backed by the same infrastructure that powers Google Search and YouTube

What is a Backend?

Every app you use has two sides:

Frontend — what the user sees and touches (buttons, screens, UI)

Backend — what happens behind the scenes (data storage, logic, security)

Imagine a To-Do app that only saves data on your phone.

1. Delete the app → all tasks gone
2. Switch to a new phone → tasks don't follow you
3. Want to share your list with a friend → not possible

A backend solves this:

1. Permanent storage
2. Sync across devices
3. Shared, accessible data

A backend is what turns a simple screen into a real, usable app.

Backend as a Service (BaaS)

Authentication

Database

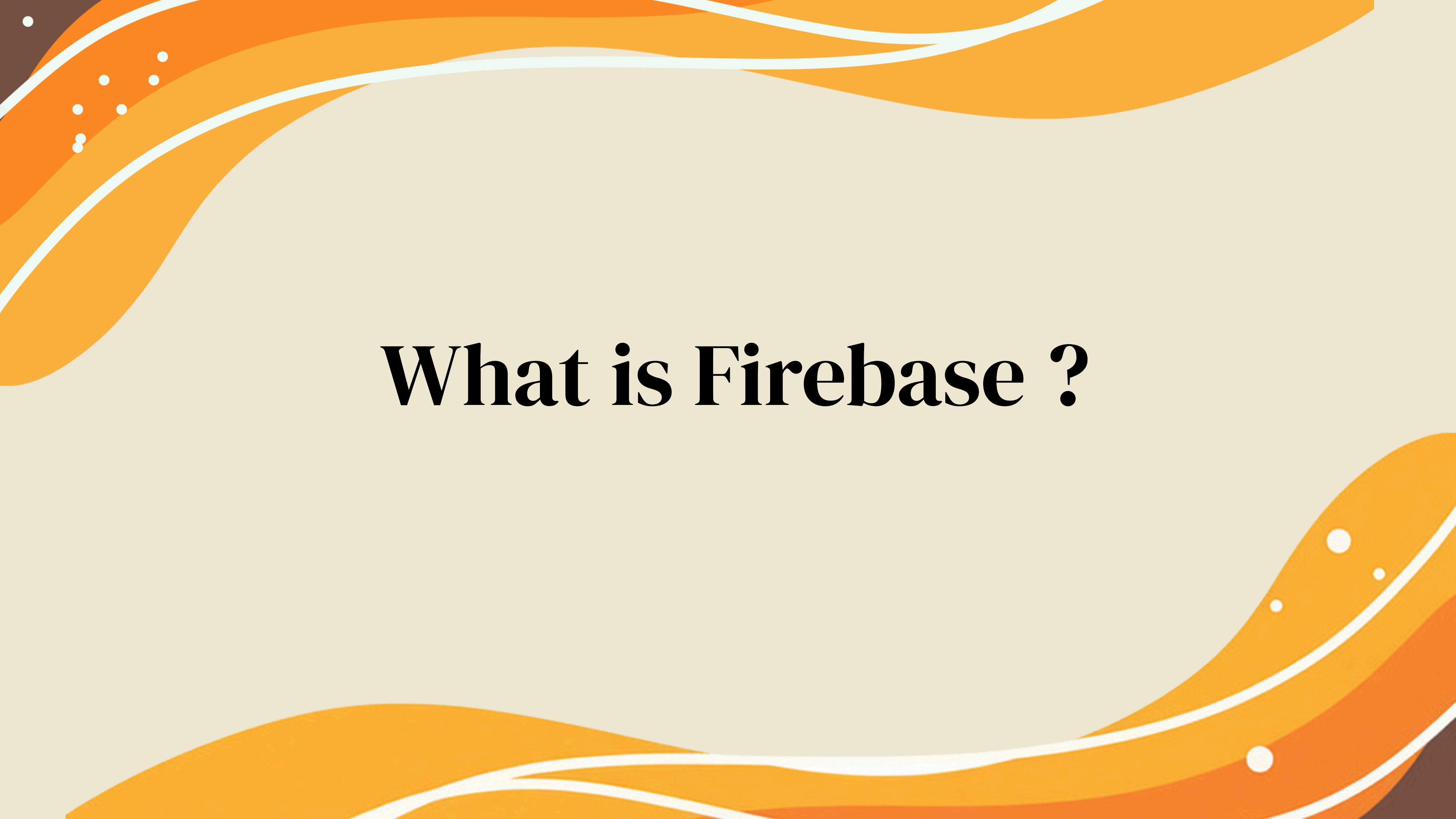
Storage

Cloud Functions

Notifications

Hosting

A backend is what turns a nice-looking screen into a real, working app.
And that's exactly what **Firestore** gives you — a backend, already built.



What is Firebase ?

Firestore

Backend-as-a-Service (BaaS) platform by Google that gives app developers a ready-made backend — no server setup required

Lets you focus on building your app instead of managing servers.

It's a collection of tools and services :

- Authentication
- Database
- Storage

.....that all work together

Instead of building a backend from scratch, you connect your app to Firestore and start using these services immediately



The Problem with Building Your Own Backend

To build a backend from scratch, you'd need to:

Rent and manage a server

Design and secure a database

Write authentication logic yourself

Handle real-time syncing (genuinely hard to get right)

Keep everything running 24/7

Google gives you all of that, ready to use:

Authentication — built in

Database — built in, with real-time sync

Hosting, storage, and more — built in

You focus on your app. Firebase handles the infrastructure.

Used by real companies: Duolingo, Alibaba, The New York Times

That's Why we use firebase



**Firestore offers a wide range of
services, grouped into three
categories**

Build

Authentication — user sign up & login

Cloud Firestore — real-time NoSQL database

Realtime Database — older real-time database option

Cloud Storage — store files (images, videos)

Cloud Functions — backend code that runs automatically

Hosting — host websites & web apps

ML Kit — machine learning features
(image/text recognition)



Release & Monitor

Crashlytics — track app crashes

Performance Monitoring — measure app speed

Test Lab — test app on real devices

App Distribution — share beta versions with testers



Engage

Cloud Messaging — push notifications

Remote Config — change app behavior without an update

A/B Testing — compare two versions of a feature

In-App Messaging — messages inside the app

Analytics — understand user behavior



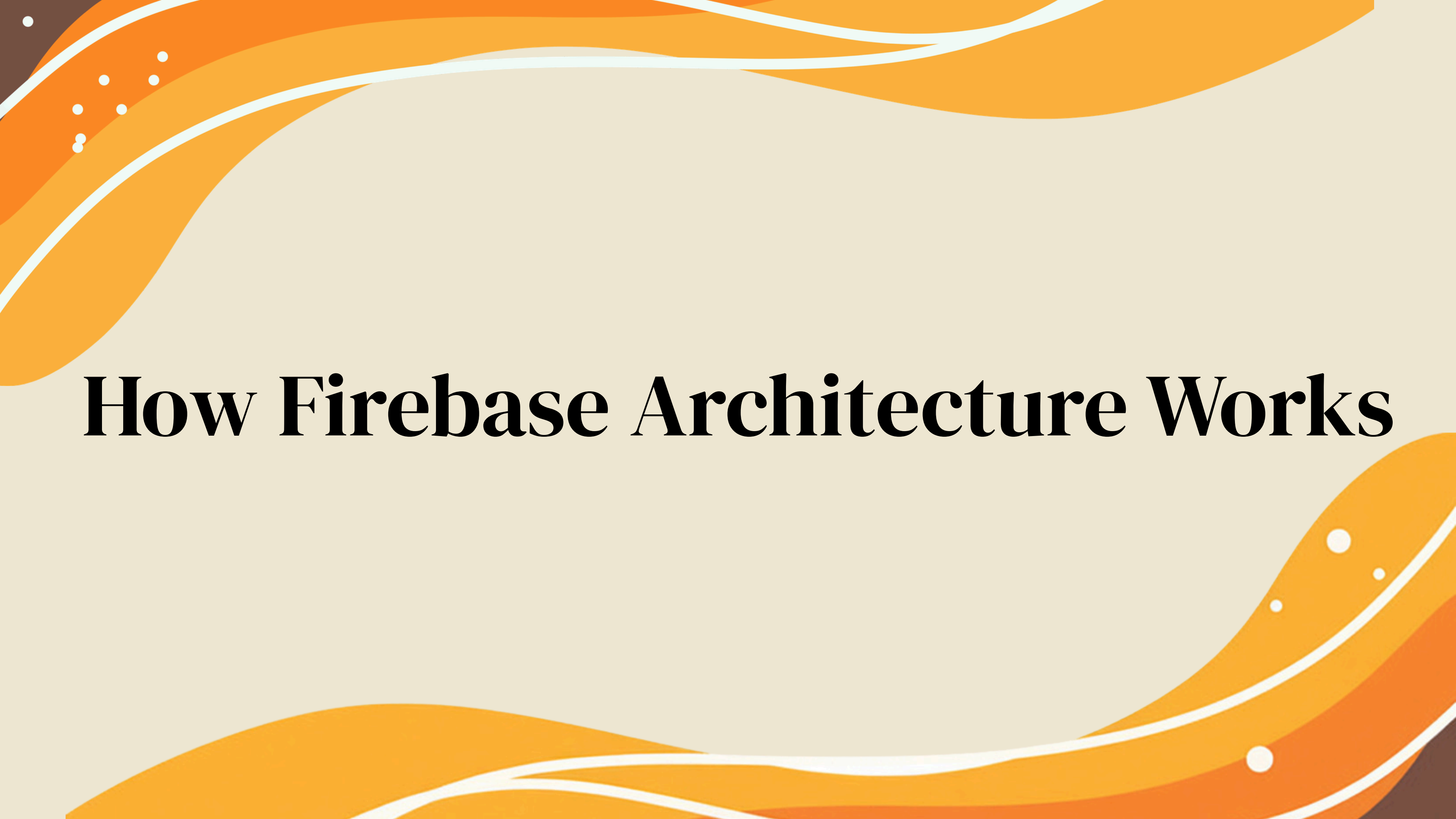
What We're Using Today

out of everything Firebase offers, today we'll focus on just two:

- **Authentication** → so users can sign up & log in
- **Cloud Firestore** → so tasks are stored and synced in real-time

Why only these three?

Because they're the foundation of almost every real app — everything else builds on top of them.



How Firebase Architecture Works

How Firebase Architecture Works





Firestore APIs

Firebase itself doesn't know Flutter — it's a separate platform
 To connect them, Google built Flutter packages ("APIs") — one for each
 Firebase service
 Each package gives you ready-made functions, so you never talk to Firebase's
 raw servers directly

Package	What it does
firebase_core	Connects your app to your Firebase project
firebase_auth	Sign up, login, sign-out
cloud_firestore	Store and read data in real-time
firebase_storage	Upload files and images



Setting Up Firebase

Setting Up Firebase — What We're About to Do

Before we touch any code, let's connect our app to Firebase.

We'll do this together, live, step by step:

1. Create a new project in Firebase Console
2. Register our Flutter app with that project
3. Connect Firebase to our Flutter project using FlutterFire CLI
4. Verify the connection works

Why do this first?

Every Firebase feature we use today (Authentication, Firestore) depends on this connection being set up correctly.

What we just did:

1. Created a Firebase project in the console
2. Registered our Flutter app (Android/iOS)
3. Ran flutterfire configure to connect the project
4. added firebase_core and initialized Firebase in main.dart

Keep this as your checklist — you'll repeat these exact steps for any future project.

Common mistake to avoid:

Forgetting to call `Firebase.initializeApp()` before `runApp()` — this causes a crash on startup.



```
void main() async {  
  
  WidgetsFlutterBinding.ensureInitialized(  
    await Firebase.initializeApp(options:  
      DefaultFirebaseOptions.currentPlatform);  
    await  
    GoogleSignIn.instance.initialize();  
    runApp(const MyApp());  
  }  
}
```



How does the app know it's you?

Authentication Who are you?

Verifying the user's identity

confirming they really are who they claim to be (via email/password, Google account, etc.)

VS

once we know who the user is, what they're allowed to see or change

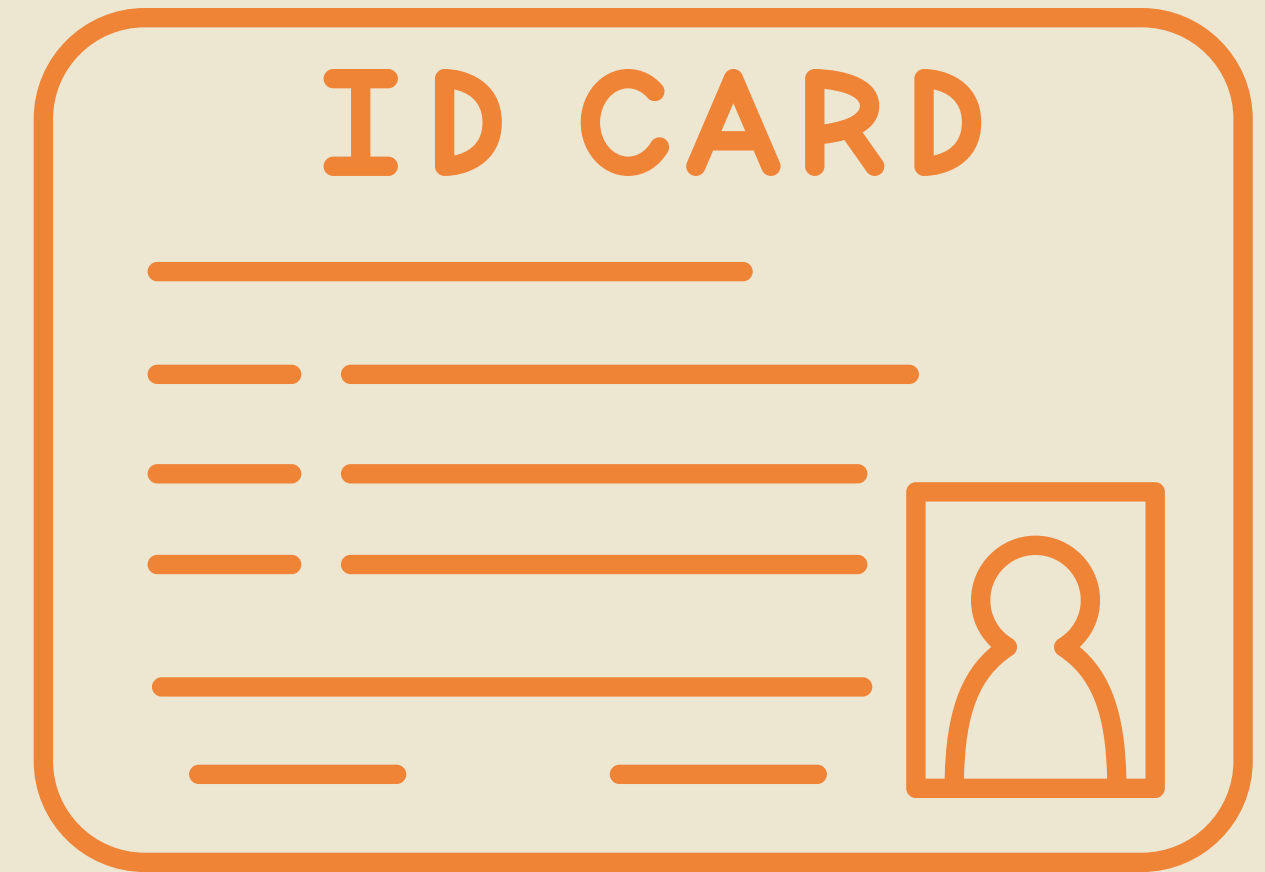
Determining permissions

What are you allowed to do?

Authorization

Sign-in Methods

- Email / Password
- Google Sign-In
- Phone
- Facebook
- Apple sign-in



Enable Sign-in Method

1. Firebase Console → Authentication → Sign-in method
2. Enable Email/Password
3. Enable Google (it'll ask for a Support email — use your own)

Step 1: Enable Authentication in the Console

- From the sidebar, go to Build → Authentication
- Click "Get Started"
- In the "Sign-in method" tab, enable the method you'll use (e.g., Email/Password)

Step 2: Connect the Flutter App to the Firebase Project

- This command will ask you to select the project you created in the Console
- It automatically generates a `firebase_options.dart` file inside your project

```
dart pub global activate flutterfire_cli  
flutterfire configure
```

Step 3: Add the Required Packages

- flutter pub add firebase_core
- flutter pub add firebase_auth

Step 4: Sign Up (register a new user)



```
final auth = FirebaseAuth.instance;

Future<void> signUp(String email, String
password) async {
  try {
    UserCredential credential = await
auth.createUserWithEmailAndPassword(
    email: email,
    password: password,
  );
  } on FirebaseAuthException catch (e) {
    print(e.message);
  }
}
```

Step 5: Login



```
Future<void> login(String email, String
password) async {
  try {
    UserCredential credential = await
auth.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
  } on FirebaseAuthException catch (e) {
    print(e.message);
  }
}
```

Step 6: Google Auth



```
Future<void>
signInWithGoogle(BuildContext context)
async {
  try {
    final googleUser = await
GoogleSignIn.instance.authenticate();
    final googleAuth =
googleUser.authentication;
    final credential =
GoogleAuthProvider.credential(
      idToken: googleAuth.idToken,
    );
    await
FirebaseAuth.instance.signInWithCredential(
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder:
(context) => Home()),
      );
    } on FirebaseAuthException catch (e)
    {
      showMySnackBar(
        msg: e.message ?? "",
        type:
AnimatedSnackBarType.error,
        context: context,
      );
    }
  }
}
```